중간 과제 공고: 팀 프로젝트를 위한 팀을 편성하는 웹 앱 만들기

1. 요약

항목	내용
과제	학생들이 팀 프로젝트를 위한 팀을 스스로 편성하는 웹 앱을 바이브 코딩으로 만듦. 남은 학생의 배정은 AI가 돕고, 앱이 AI의 답을 규칙으로 검사함
배점	30점
범위	1~6장(PRD와 구현 계획, 검증 규칙, API와 키 관리, Streamlit 앱, Supabase DB, 로그인, 배포)
공고	10/16(금)
마감	11/2(월) 23:59
채점 기간	11/3(화)~11/4(수). 11/3에 시작하고, 하루에 끝나지 않으면 11/4까지 이어서 채점함. 이 기간에 DB(Supabase)가 일시정지되어 있으면 안 됨
채점 방법	Streamlit에 배포한 앱에서, 공개하지 않은 채점 시나리오 3개(쉬움, 중간, 어려움)를 실행함. 부록의 시나리오 3개는 연습용이며 채점에 쓰지 않음
제출물	깃허브 저장소 주소, 배포 주소, 관리자 비밀번호(e-class 제출 칸에만), 저장소 압축 파일
개인 과제	혼자 만듦. 클로드 사용은 허용하고 권장함
시연 동영상	https://youtu.be/ti-mDlKIfqg. 시범 앱(이 공고만 보고 클로드로 미리 만들어 본 앱)으로 연습용 쉬움 시나리오를 처음부터 끝까지 해 보고, 중간 시나리오를 시작하는 모습까지 보여 줌(12분). 동영상에서 "채점 시나리오"라고 부른 3개는 공고가 바뀌어 연습용 시나리오가 되었음. 채점은 공개하지 않은 다른 시나리오로 함

- 기능마다 "쉬운 해결"과 "어려운 해결"을 함께 안내함. 쉬운 해결은 기능에 따라 0점이거나 일부 점수를 받고, 어려운 해결을 하면 더 큰 점수를 받음(3장 표의 점수 열)
- 진도가 늦어져서 8주차까지 6장 수업이 끝나지 않을 수도 있으니, 미리 노트를 공부하여 과제를 수행하기 바람

2. 앱이 하는 일

2.1 등장인물

- 이 공고의 "학생"은 앱을 쓰는 가상 학생(명단 CSV의 학생)임. 과제를 내는 수강생은 앱을 만드는 사람이며, 시험할 때 가상 학생과 관리자로 번갈아 로그인해 확인해야 함

사용자	하는 일
학생	학번과 비밀번호로 로그인함. 팀 목록과 팀원, 팀원의 주 역할과 보조 역할을 봄. 팀에 지원함. 내 팀에 들어온 지원에 찬성 또는 거부를 누름
관리자(교수)	학번 `admin`으로 로그인함. 명단을 올려 처음 상태를 만듦. 모집을 마감함. 마감 뒤 남은 학생의 배정안을 받아 확인하고 확정함

- 관리자 계정은 앱이 시작할 때 비밀 값 `ADMIN_PASSWORD`(4.1절)로 만들거나 비밀번호를 갱신함. 명단을 다시 올려도 관리자 계정은 남김

2.2 명단과 역할

- 학생 명단은 관리자가 CSV 파일로 올림(2.6절). 학생의 비밀번호는 모두 `1234`임
- 실제 학생의 이름과 학번은 쓰지 않음. 교수가 나눠 주는 가상 명단만 씀
- 학생의 역할은 6개임: 기획자, 웹 프로그래머, 프론트엔드 디자이너, 테스터, 보고서 작성자, 발표자
- 학생마다 주 역할 1개와 보조 역할 0~2개가 명단 CSV에 정해져 있음. 학생이 바꾸지 않음

2.3 팀 규칙

규칙	내용
팀	관리자가 명단을 올릴 때, 시나리오에서 정해 준 팀 수를 넣어 그 수만큼 팀을 만듦(팀1, 팀2, …). 팀 수는 부록의 시나리오 제목에 적혀 있음
인원	마감 뒤 확정한 팀 구성에서는 모든 팀이 2~4명이어야 함(빈 팀 없음)
주 역할 중복	주 역할이 동일한 학생이 한 팀에 2명 이상이면 안 됨
필수 포함 역할	확정한 팀 구성에서, 모든 팀에는 기획자 역할 학생과 웹 프로그래머 역할 학생이 서로 다른 사람으로 1명씩 있어야 함. 주 역할이나 보조 역할로 맡을 수도 있음

- 주 역할은 팀 안의 중복을 따지는 기준이고, 보조 역할은 필수 포함 역할을 맡을 수 있는지 따질 때만 씀
- 예: 주 역할이 테스터이고 보조 역할이 기획자인 학생이 그 팀의 기획자를 맡을 수도 있음
- 지원과 투표를 하다 보면 필수 포함 역할을 지킬 수 없는 팀이 생길 수 있음. 예를 들어 웹 프로그래머를 맡을 수 있는 학생(주 역할이나 보조 역할)이 한 명도 없이 4명이 다 찬 팀이 있으면, 이 팀에는 더는 학생을 넣을 수 없고 마감 뒤 배정에서도 기존 팀원을 옮기지 않으므로 끝까지 웹 프로그래머를 채우지 못함

2.4 지원과 투표

상황	규칙
빈 팀에 지원	투표할 팀원이 없으므로 바로 승인됨
팀원이 있는 팀에 지원	대기. 그 팀의 팀원 모두가 투표함. 투표할 사람은 투표하는 때의 팀원임(예: 내가 지원한 뒤에 다른 지원자가 먼저 승인되어 팀원이 되면, 그 사람도 내 지원에 투표함)
모든 팀원이 찬성	승인됨. 그 팀의 팀원이 됨
한 명이라도 거부	다른 팀원의 투표를 기다리지 않고 바로 거절됨
아직 투표하지 않은 팀원이 있음	대기
승인 직전	모든 팀원이 찬성해 승인하려는 때에 규칙을 다시 검사함. 같은 팀에 대기 중인 지원이 여럿이면 앞의 지원이 먼저 승인되어 팀이 바뀌기 때문임. 승인 직전 검사에서 규칙(정원, 주 역할 중복, 필수 포함 역할 자리)을 어기게 되면 그 지원은 거절됨. 이 재검사는 3장 "같은 팀에 대기 중인 지원이 여럿"의 어려운 해결임. 그 전에는 대기 중인 지원을 미리 거절하지 않음

상황	규칙
한 학생의 지원	지원한 팀의 결과(승인 또는 거절)가 나오기 전에는 다른 팀에 지원할 수 없음. 결과가 나온 뒤에는 다른 팀에 다시 지원할 수 있음
팀이 꽉 참	4명인 팀에는 지원할 수 없음
같은 주 역할	주 역할이 중복되는 팀원이 있는 팀에는 지원할 수 없음
이미 팀이 있음	다른 팀에 다시 지원할 수 없음
막힌 지원	지원 단추를 누른 순간 앱이 규칙을 검사해 지원을 접수하지 않는 것. 이유는 팀이 꽉 참, 같은 주 역할의 팀원이 있음, 이미 팀이 있거나 결과를 기다리는 지원이 있음, 모집이 마감됨, 필수 포함 역할을 맡을 학생의 자리가 모자람(3장의 어려운 해결을 한 앱)임. 결과 안내는 "막힘"으로 시작하고 이유를 함께 보여 줌(예: "막힘: 같은 주 역할의 팀원이 있음"). 팀원의 투표 없이 끝나며, DB에 기록하지 않아도 됨
막힘과 거절의 차이	막힘은 지원을 아예 접수하지 않은 것임. 거절은 접수되어 대기 중이던 지원이 팀원의 거부, 승인 직전 검사, 모집 마감으로 떨어진 것임(결과 안내는 "거절"로 시작함)
투표 바꾸기	한 번 투표하면 바꿀 수 없음

2.5 마감과 배정

1. 관리자가 "모집 마감"을 누름. 그 뒤로는 지원과 투표를 할 수 없음
2. 마감 때 대기 중인 지원은 거절됨. 1명뿐인 팀의 학생은 팀에서 빠져 "팀이 없는 학생"이 됨
3. 관리자가 "AI 배정"을 누르면, 앱이 AI에게 지금의 팀 구성, 팀이 없는 학생과 그 역할, 2.3절의 규칙을 보내 배정안을 받음. AI는 오픈라우터의 텍스트 생성 모델을 씀(4.3절). 배정은 팀이 없는 학생만 팀에 넣으며, 이미 팀에 있는 학생은 옮기지 않음
4. 앱이 배정안을 다음 다섯 가지로 검사함: 팀이 없는 학생이 빠짐없이 배정됨, 한 학생은 한 팀에만 배정됨, 모든 팀이 2~4명(2.3절 인원), 팀 안에서 주 역할 중복 없음(2.3절), 필수 포함 역할 충족(2.3절)
5. AI가 응답하지 않거나, 빈 응답이나 오류를 내거나, 배정안이 4항의 검사를 통과하지 못하면, 앱이 자동으로 2차 모델, 3차 모델 순으로 다시 배정함. 모델 하나를 부른 때부터 응답을 받을 때까지 120초가 넘으면 끊고 다음 모델로 넘어감
6. 3차 모델까지 실패하면 앱이 스스로 배정함(3장의 쉬운 해결과 어려운 해결)
7. 검사를 통과한 배정안만 관리자에게 보여 줌(학생의 이름과 역할, 배정된 팀). 관리자가 확정하면 DB에 저장함
8. 규칙을 모두 지키는 배정이 없으면, 앱은 배정안을 만들지 않고 관리자 화면에 "규칙을 지키는 배정이 없음"과 그 이유(예: 팀3에 웹 프로그래머를 맡을 학생이 없음)를 보여 줌. 이때는 AI를 부르지 않아도 됨

2.6 명단 올리기

- 관리자 화면에서 명단 CSV 파일과 팀 수를 넣으면 앱 상태가 초기화됨: 지원, 투표, 팀원, 마감 표시를 모두 지우고, 명단의 학생과, 넣은 팀 수만큼의 빈 팀을 만듦
- 채점자는 이 기능으로 시나리오마다 명단을 바꿈. 이 기능이 없으면 시나리오로 채점할 수 없음
- CSV 형식(UTF-8)
- 나눠 주는 CSV는 엑셀에서 한글이 깨지지 않도록 BOM이 붙은 UTF-8임. 앱은 BOM이 있는 파일도 읽어야 함. 클로드에게 "CSV를 utf-8-sig로 읽게 해 줘"라고 요청함

```
student_no,name,main_role,sub_roles
2026001,가나다,기획자,보고서 작성자
2026002,라마바,웹 프로그래머,
2026003,사아자,테스터,기획자;발표자
```

- `sub_roles`가 2개면 `;`로 구분하고, 없으면 마지막 쉼표 뒤를 비워 둠(위 예의 `2026002` 줄). 쉼표를 지우면 열이 3개가 되어 CSV가 틀림

2.7 화면 이름 표

채점은 자동 채점 프로그램이 Streamlit에 배포한 앱의 화면을 자동으로 조작해서 진행함. 자동 채점 프로그램은 화면 요소를 Streamlit의 `key`로 찾으므로, 아래 표의 요소마다 표에 적힌 `key`를 그대로 지정해야 함. 화면에 보이는 글자는 자유롭게 써도 됨

화면 요소	Streamlit 요소	`key`
학번 입력칸	`st.text_input`	`login_student_no`
비밀번호 입력칸	`st.text_input`	`login_password`
로그인 단추	`st.button` 또는 `st.form_submit_button`	`login_submit`
로그인한 사람 표시(이름이 보임)	`st.container`	`whoami`
단추를 누른 결과 안내	`st.container`	`result`
팀 하나의 표시(팀원의 이름이 보임)	`st.container`	`team_1`, `team_2`, …
팀에 지원하는 단추	`st.button`	`apply_1`, `apply_2`, …
지원에 찬성하는 단추	`st.button`	`vote_yes_{지원자 학번}` 예: `vote_yes_2026306`
지원에 거부하는 단추	`st.button`	`vote_no_{지원자 학번}`
명단 CSV 올리기	`st.file_uploader`	`admin_roster_file`
팀 수	`st.number_input`	`admin_team_count`
명단으로 처음 상태 만들기 단추	`st.button`	`admin_start`
모집 마감 단추	`st.button`	`admin_close`
팀이 없는 학생 표시(이름이 보임)	`st.container`	`unassigned`
AI 배정 단추	`st.button`	`admin_assign`
AI 배정 진행 안내	`st.container`	`assign_status`
배정안 확정 단추	`st.button`	`admin_confirm`
새로 고침 단추(3장 "다른 사람의 변경 보기"를 쉬운 해결로 만든 앱만)	`st.button`	`refresh`

- 다른 사람의 변경 보기를 쉬운 해결(새로 고침 단추)로 만든 앱은 `refresh` 단추를 학생 화면과 관리자 화면에 모두 두어야 함. 채점 프로그램은 화면을 다시 열지 않고, 동작하기 전에 이 단추가 있으면 먼저 누름. 이 단추가 없으면 다른 학생의

지원이나 투표가 화면에 나타나지 않아 채점이 멈출 수 있음

- 한글 `key`는 채점 프로그램이 찾지 못하므로, `key`에는 영문, 숫자, 밑줄만 씀
- `result`는 단추를 누른 사람의 화면에 보임. 지원 단추를 누르면 그 지원의 결과를, 찬성이나 거부 단추를 누르면 그 지원의 지금 상태(승인, 대기, 거절)를 보여 줌. 한 단계에 동작이 여럿이면 채점 프로그램은 동작마다 `result`를 확인함
- 마감 때 대기 지원이 거절되고 1명뿐인 팀의 학생이 빠진 결과는 관리자 화면의 `unassigned`(팀이 없는 학생 표시)로 확인함
- 규칙을 지키는 배정이 없을 때의 안내(2.5절 8항)는 `assign_status` 안에 보여 줌
- 관리자 단추(명단 올리기, 모집 마감, AI 배정, 확정)의 `result`도 해냈으면 "승인", 하지 못했으면 "막힘"으로 시작함. 예: "승인: 모집을 마감함"
- `unassigned`에는 팀이 없는 학생의 이름을 보여 주면 됨
- 다른 사람의 변경을 자동으로 다시 표시하는 앱은 5초 이하의 간격으로 다시 그림. 채점 프로그램은 단추가 나타나기를 7초까지 기다림
- `result`의 안내는 "승인", "대기", "거절", "막힘" 가운데 하나로 시작함. 예: "막힘: 같은 주 역할의 팀원이 있음"
- 2.8절의 경우(이미 팀이 있음, 결과를 기다리는 지원이 있음, 모집 마감)에는 지원 단추를 숨기거나 비활성화해야 함. 그 밖에 막히는 지원(같은 주 역할 등)은 단추를 누른 뒤 "막힘" 안내로 막아도 되고, 단추를 숨겨도 됨. 채점 프로그램은 단추가 없거나 비활성이면 "막힘"으로 봄
- 팀 번호(`team_1`의 1)는 관리자가 명단을 올릴 때 만든 팀1, 팀2, …의 번호이며, 시나리오의 팀1, 팀2와 같음
- `st.container`에 `key`를 지정하는 방법: 클로드에게 "단추를 누른 결과 안내는 key가 result인 st.container 안에 보여 줘"라고 요청함
- 이 표에 적힌 Streamlit 요소(입력칸, 단추, 표시 영역)에 Streamlit `key`를 표와 다르게 지정했거나 지정하지 않아, 자동 채점 프로그램이 그 요소를 찾지 못하고 멈추면, 그 뒤의 채점 항목은 채점자가 직접 조작해 확인함. 이와 별도로 6.3절(감점)에 따라 5점을 감점함

2.8 학생 화면

- 팀 목록에서 로그인한 학생의 팀을 구별되게 표시해야 함(예: 팀 이름 옆에 "내 팀"). 팀이 없으면 "아직 팀이 없음"을 보여 줌(이 표시는 채점하지 않음)
- 이미 팀에 속했거나, 결과를 기다리는 지원이 있거나, 모집이 마감되었으면 지원 단추를 숨기거나 비활성화해야 함
- 이미 투표한 지원에 대해서는 그 학생의 찬성, 거부 단추를 숨기거나 비활성화하고, "내 투표: 찬성"처럼 보여 줘야 함

3. 기능별 쉬운 해결과 어려운 해결

이 장의 기능은 모두 앱에 만들어야 하는 것이며, 기능마다 쉬운 해결과 어려운 해결 가운데 하나를 골라 만듦

기능	쉬운 해결	어려운 해결	점수(쉬운 해결, 어려운 해결)
다른 사람의 변경 보기	새로 고침 단추	바뀐 부분만 자동으로 다시 표시 (Streamlit `st.fragment`)	0.5점, 1점
필수 포함 역할을 지킬 수 없는 팀	배정 단계에서 "규칙을 지키는 배정 없음"을 안내함	지원 단계에서, 필수 포함 역할을 채울 자리가 모자라게 되는 지원은 막음	1.5점, 7점
같은 팀에 대기 중인 지원이 여럿	지원할 때만 규칙을 검사하고, 승인할 때는 다시 검사하지 않음. 앞의 지원이 먼저 승인되어 팀이 바뀌면 규칙을 어길 수 있음	승인 직전에 규칙을 다시 검사함. 처리 순서는 지원한 순서가 아니라 투표가 끝난 순서임	0점, 3점

기능	쉬운 해결	어려운 해결	점수(쉬운 해결, 어려운 해결)
마지막 자리에 두 명이 동시에 들어감	앱이 인원을 센 뒤 넣음	DB의 유일성 제약으로 먼저 도착한 학생만 기록함	0점, 0.5점
AI 응답이 늦을 때	AI 요청 함수에 시간 제한 값(예: 120초)을 줌. 이 값은 응답이 시작될 때까지만 재는 경우가 많아, 응답이 천천히 오면 120초를 넘길 수 있음	모델마다 120초의 전체 대기 시간을 재서 끊고 다음 모델로 넘어감	0.5점, 1점
3차 모델까지 실패	자리가 남은 팀에 무작위로 넣음. 2.5절 4항의 검사를 통과하지 못하면 그 사실을 함께 보여 줌	앱이 2.5절 4항의 검사 다섯 가지를 모두 통과하는 배정을, 모든 경우를 따져 끝까지 찾음	채점 때 AI가 3차까지 실패하면, 쉬운 해결은 확정 뒤 규칙 점수(최대 6.5점)를 잃을 수 있음
AI 대기 중 화면	"AI가 생각하는 중" 안내	몇 차 모델의 응답을 기다리는 중인지 보여 줌	0.5점, 1점
DB(Supabase)가 일시정지되지 않게	제출 마감 날(11/2) 배포한 앱을 써서 DB 요청을 만듦	외부 예약 서비스로 매일 DB에 질의를 보냄	0점, 0.5점

- 점수 열의 점수는 6.1절의 묶음 점수(30점) 안에 들어 있음. 쉬운 해결은 그 항목에서 0점이거나 부분 점수를 받고, 어려운 해결은 그 항목의 점수를 다 받음
- 들어 있는 묶음: 다른 사람의 변경 보기는 쉬움 시나리오, 필수 포함 역할을 지킬 수 없는 팀, 같은 팀에 대기 중인 지원이 여럿, AI 응답이 늦을 때, AI 대기 중 화면은 어려움 시나리오, 마지막 자리에 두 명이 동시에 들어감과 DB 일시정지는 시나리오 밖임. "3차 모델까지 실패"의 6.5점은 중간 시나리오(2.5점)와 어려움 시나리오(4점)의 확정 뒤 규칙 점수를 더한 것임
- 필수 포함 역할 자리 계산: 지원자가 들어간 뒤 남는 자리 수가, 그 팀에서 아직 채우지 못한 필수 포함 역할 수보다 적으면 지원을 막아야 함. 기획자와 웹 프로그래머는 서로 다른 학생이 맡아야 하므로, 한 학생이 둘 다 할 수 있어도 하나로만 셈. 예: 주 역할이 기획자이고 보조 역할이 웹 프로그래머인 학생 혼자만 있는 팀은 필수 포함 역할 2개 가운데 1개만 채운 것임. 또 다른 예: 테스터와 발표자만 있는 팀(2명)에 보고서 작성자가 지원하면, 남는 자리 1개로 필수 포함 역할 2개를 채울 수 없으므로 막음
- 마지막 자리에 두 명이 동시에 들어가는 일은, 예를 들어 빈 팀에 두 학생이 같은 순간 지원하면 생길 수 있음. 채점 프로그램은 단추를 차례로 누르므로 채점에서 이 현상을 직접 만들지는 않고, DB 유일성 제약이 있는지만 `schema.sql`로 확인함
- 앱이 잠들면 채점자가 깨우므로 학생이 깨워 둘 필요는 없음. DB(Supabase)는 1주일 동안 요청이 적으면 일시정지되고 채점자가 다시 켤 수 없으므로, 4.4절의 방법으로 채점 기간 전 1주일 안에 DB를 써야 함
- 로그인은 학번과 비밀번호로 함. 비밀번호는 해시로 저장해야 함. 구글 로그인(`st.login`)은 채점자가 학생 여러 명으로 로그인할 수 없으므로 이 과제에서 쓰지 않음

4. 꼭 지킬 것

4.1 키 관리

- 오픈라우터 키, DB 주소, 관리자 비밀번호는 `.env` 파일에 두고, 배포한 앱에서는 이 값들을 Streamlit Secrets에 둠
- `.env` 파일의 내용은 다음과 같은 꼴임. 값은 각자의 것으로 바꾸고, Streamlit Secrets에도 같은 이름으로 넣음(4.4절)

```
OPENROUTER_API_KEY="sk-or-v1-자기 키"
DATABASE_URL="Supabase에서 복사한 DB 주소"
```

```
ADMIN_PASSWORD="정한 관리자 비밀번호"
```

- `.env`는 앱 파일(예: `streamlit_app.py`)과 같은 폴더에 둠
- 파일 이름은 정확히 `.env`여야 함. 컴퓨터의 편집기가 `.env.txt`로 저장하면 `.gitignore`가 걸러 내지 못해 키가 깃허브에 올라감
- `.env`를 두 번 눌러 열지 않음. 컴퓨터에 따라 터미널이 이 파일을 명령으로 실행할 수 있음
- 클로드에게 키 값을 화면에 출력하지 말라고 지시해야 함. 키가 있는지 확인할 때는 "있음, 없음"만 출력하게 함
- 키나 관리자 비밀번호를 코드에 직접 적거나 깃허브에 올리면(커밋 기록 포함) 감점함

4.2 DB

- 6장 실습에서 만든 Supabase 프로젝트를 재사용함. 6장 실습 전이면 노트 6장의 절차로 프로젝트를 먼저 만들고, 그 프로젝트를 6장 실습에도 씀. 새 프로젝트를 만들지 않음(무료 프로젝트는 2개까지만 만들 수 있음)
- 이 과제의 테이블 이름은 모두 `tf_`로 시작함. 6장 실습의 테이블과 이 과제의 테이블이 같은 DB(같은 Supabase 프로젝트)에 함께 들어가므로, 이름 앞에 `tf_`를 붙여 구분함
- 최소 참고안(5장)을 그대로 써도 되고, 고쳐서 늘려도 됨. 예: 열을 더함(지원 시각, 거절 이유), 테이블을 더함(AI 호출 기록. 시범 앱은 `tf_ai_logs` 테이블을 더했음)
- 질의(쿼리)를 할 때마다 DB 연결을 새로 만들면 오래 걸림. 예: 화면을 한 번 그릴 때 질의를 10번 하는데, 질의마다 연결을 새로 만들면 연결에만 0.5초씩 걸려 화면이 5초 늦어짐. 그래서 연결은 한 번만 만들고, 그 연결을 사용해 여러 번의 질의를 수행해야 함. 화면이 몇 초씩 느리면 클로드에게 "DB 연결을 한 번만 만들고 다시 쓰게 고쳐 줘"라고 요청함

4.3 AI 모델

- 오픈라우터의 텍스트 생성 모델 3개를 1차, 2차, 3차로 정해 씀. 모델은 각자 고름
- 무료 텍스트 생성 모델을 쓰기를 권함. 무료 모델은 이름이 `:free`로 끝남. 유료 모델을 써도 감점하지 않으나, 요금은 각자 부담함
- 1차, 2차, 3차로 정하는 이유: 무료 모델은 응답이 늦거나(120초 넘음), 사용량이 많다며 거절하거나(HTTP 429), 빈 응답을 내는 일이 잦음. 시범 앱을 시험했을 때 일곱 번 가운데 세 번이 이렇게 실패했음. 모델을 하나만 쓰면 채점 때 배정이 실패할 수 있으므로, 앞 모델이 실패하면 다음 모델로 넘어가게 해야 함
- 무료 모델로 자동 연결하는 라우터(`openrouter/free`)는 쓰지 않음. 다음 모델로 넘어가는 처리를 직접 만드는 것이 이 과제의 일부이기 때문임
- 추론 모델(답하기 전에 오래 생각하는 모델)은 응답 길이 한도(`max_tokens`)를 생각하는 데 다 써서 빈 응답을 낼 수 있음. 빈 응답도 실패로 처리하고 다음 모델로 넘어감. 빈 응답을 줄이려면 클로드에게 "오픈라우터 요청에 reasoning effort를 low로 주고 max_tokens를 넉넉히 늘려 줘"라고 요청함(시범 앱에서 이 방법으로 빈 응답을 줄였음)
- 무료 모델은 계정마다 하루 50회, 분당 20회까지만 부를 수 있음. 실패한 호출도 횟수에 들어감. 채점 기간(11/3~11/4)에는 채점자가 학생의 키로 AI를 부르므로, 그 이틀 동안은 하루 한도 가운데 30번 이상을 남겨 두어야 함. 연습용 시나리오 3개를 한 번 돌리는 데 AI 호출이 3~9번 듦. 무료 모델은 밤에 더 자주 실패했음(10/15 밤 시험에서 무료 모델 11개 가운데 배정 안을 낸 것은 3개)
- 무료 모델은 응답에 1~3분이 걸리기도 하고, 채점 때 쓸 수 없기도 함. 그래서 3차까지 실패하면 앱이 스스로 배정해야 함
- 채점은 AI가 낸 배정의 내용이 아니라, 확정한 팀 구성이 규칙을 지키는지와 실패를 처리했는지를 확인함. 실패 처리는 모델마다 120초 제한, 2차와 3차 모델로 넘어감, 3차까지 실패하면 앱이 스스로 배정함을 화면 기록이나 코드로 확인함

4.4 배포, 앱 잠자기, DB 일시정지

- Streamlit Community Cloud에 배포함. 채점은 Streamlit에 배포한 앱으로만 함

- 깃허브 저장소는 비공개로 만듦. 마감 전에 다른 수강생이 코드를 볼 수 없게 하기 위함임
- 비공개 저장소로 배포하려면, 배포하기 전에 Streamlit이 비공개 저장소를 읽도록 허락해야 함: Streamlit의 Workspace settings > Linked accounts에서 Private access 옆의 "Connect here"를 누름
- 비공개 저장소로 배포한 앱은 처음에 비공개로 설정됨. 배포한 뒤 앱의 Settings > Sharing에서 "Who can view this app"을 "This app is public and searchable"로 바꾸고 "Save changes"를 누름
- 바꾼 뒤 로그인하지 않은 브라우저(시크릿 창)로 배포 주소를 열어 로그인 화면이 보이는지 확인함. 채점 프로그램은 로그인하지 않은 브라우저로 앱을 엶
- 배포 화면의 Branch 칸에는 자기 저장소의 브랜치 이름(`master` 또는 `main`)을 넣음
- Main file path 칸에는 앱 파일의 이름(예: `streamlit_app.py`)을 넣음
- Secrets 칸은 `이름 = "값"` 꼴로 씀. 브라우저 자동 완성이 예전 값을 채우면 지우고 다시 써야 함
- 배포한 앱은 12시간 동안 방문이 없으면 잠자기 상태가 됨. Supabase 무료 프로젝트는 1주일 동안 DB 요청이 적으면 일시정지됨
- 앱이 잠들어 있으면 채점자가 깨워서 채점함(감점하지 않음). 잠든 앱은 로그인하지 않은 사람도 깨울 수 있음
- DB(Supabase)는 프로젝트 주인만 다시 켤 수 있음. 채점 기간(11/3~11/4)에 DB가 일시정지되어 있으면 채점자가 앱을 쓸 수 없어 채점할 수 없고, 시나리오 점수는 0점이 됨(6.3절 16번)
- DB가 일시정지되지 않게 하는 방법(Supabase 문서 「Project Pausing」에 따름)
- Supabase는 무료 프로젝트에서 1주일 동안 DB 요청이 적으면 일시정지함(정확한 기준은 밝히지 않음). 하루 몇 번의 DB 요청이면 보통 일시정지되지 않음
- 마감 날(11/2)에 배포한 앱에 로그인해 명단을 올리고 지원과 투표를 몇 번 해 봄. 앱을 쓰는 것이 곧 DB 요청임
- Supabase는 일시정지하기 약 1주일 전에 예고 메일을 보냄. 예고 메일을 받으면 Supabase 대시보드에서 그 프로젝트를 열거나 앱을 써서 DB 요청을 만듦
- 이미 일시정지되었으면 Supabase 대시보드에서 그 프로젝트를 골라 다시 켬. 채점 기간 전에 해야 함
- 자동으로 막으려면 3장 표의 어려운 해결(외부 예약 서비스로 매일 DB 요청)을 씀
- 배포 서버의 시각은 세계 표준시(UTC)임. 화면에 시각을 보여 주려면 한국 시각으로 바꿔서 보여 줘야 함

5. DB 최소 참고안

테이블	열	뜻
`tf_students`	`id`, `student_no`(학번, 유일), `name`, `main_role`, `password_hash`, `is_admin`	학생과 관리자
`tf_student_sub_roles`	`student_id`, `role`	보조 역할. 학생 1명에 0~2행
`tf_teams`	`id`, `name`(유일)	팀
`tf_members`	`team_id`, `slot_no`(1~4), `student_id`(유일)	팀원. `(team_id, slot_no)`를 기본 키로 둠
`tf_applications`	`id`, `team_id`, `student_id`, `status`(대기, 승인, 거절), `reason`, `created_at`	지원
`tf_votes`	`application_id`, `voter_id`, `vote`(찬성, 거부)	투표. `(application_id, voter_id)`를 기본 키로 둠
`tf_settings`	`key`, `value`	마감 여부 등

- `tf_members`의 `(team_id, slot_no)`와 `student_id`에 설정한 유일성 제약이, 3장의 "먼저 도착한 학생만 기록"과 "한 학생은 한 팀에 소속"을 DB에서 지켜 줌
- `slot_no`에는 `check (slot_no between 1 and 4)`를 설정해 다섯 번째 자리를 막음
- `main_role`과 `role`에는 `check (... in ('기획자', '웹 프로그래머', ...))`를 설정해 6개 역할 밖의 값을 막음
- 테이블을 만드는 SQL을 저장소의 `schema.sql` 파일로 남겨야 함. 채점자는 학생의 Supabase를 볼 수 없으므로, 이 파일로 DB 유일성 제약(3장 표의 "마지막 자리에 두 명이 동시에 들어감"), 테이블 이름(6.3절 13번), 비밀번호 해시 열(6.3절 10번)을 확인함. 파일이 없으면 유일성 제약의 점수(0.5점)를 받지 못하고, 테이블 이름과 비밀번호 해시는 확인할 수 없으므로 6.3절 13번과 10번을 적용함
- 클로드에게 다음과 같이 요청해 만듦: "Supabase에 만든 tf_ 테이블을 만드는 SQL을 저장소의 schema.sql 파일로 남겨 줘. 앱이 실제로 쓰는 테이블, 열, 제약과 같은지 확인하고, 다르면 알려 줘."
- Supabase 화면의 SQL 편집기에서 테이블을 직접 만들었다면, 그때 쓴 SQL도 `schema.sql`에 넣어야 함

6. 채점

6.1 점수 배분

구분	점수	내용
쉬움 시나리오	4	로그인, 지원, 투표, 마감, 배정과 확정의 기본 흐름, 다른 사람의 변경 보기
중간 시나리오	8	규칙(주 역할 중복, 거부, 한 번에 한 팀, 마감 처리)과 배정안 검사
어려움 시나리오	12	승인 직전 재검사, 필수 포함 역할 자리 지키기, 보조 역할을 쓴 배정, AI 실패 처리와 대기 안내
시나리오 밖	6	PRD와 구현 계획, `README.md`의 사례(7.1절 5~7항), 키 관리, DB 유일성 제약, DB 일시정지 막기(3장의 어려운 해결)

- 어려움 시나리오는 잘 만든 앱을 가리기 위한 것임. 쉬움과 중간 시나리오는 30점 가운데 12점임. 시험하며 찾은 사례를 적는 `README.md`의 7.1절 5~7항은 4점임
- 쉬움, 중간, 어려움은 공개하지 않은 채점 시나리오의 묶음임. 부록의 연습용 시나리오와 난이도가 같음
- 해결 방법에 따라 달라지는 점수는 3장 표의 점수 열에 있음

6.2 채점 절차

1. 채점 프로그램이 배포 주소를 열고 관리자로 로그인해 시나리오의 명단을 올림
2. 프로그램이 가상 학생마다 브라우저를 따로 열어 채점 시나리오의 단계를 차례로 실행함. 화면 요소는 2.7절의 `key`로 찾음
3. 단계마다 결과 안내(`result`)의 첫 단어(승인, 대기, 거절, 막힘)가 채점 시나리오의 기대 결과와 같은지 확인함
4. 앞 단계의 결과가 기대 결과와 달라도, 채점 프로그램은 이어 갈 수 있는 단계를 끝까지 이어 감. 앞 단계의 결과 때문에 할 수 없게 된 단계(예: 팀에 들어가지 못한 학생의 투표)는 그 단계의 점수만 받지 못함
5. 마지막에 브라우저를 새로 열어, 확정한 팀 구성이 남아 있고 규칙을 지키는지 검사함
6. 프로그램이 멈추면 채점자가 그 단계부터 직접 조작해 이어 감
7. 채점은 공개하지 않은 채점 시나리오 3개(쉬움, 중간, 어려움)로 함. 부록의 시나리오는 연습용이며 채점에 쓰지 않음. 채점 시나리오의 난이도는 연습용 시나리오와 같게 맞추고, 이 공고에 적힌 규칙으로만 시험함

8. 채점 시나리오는 연습용 시나리오와 비교해 학생 수(8~16명), 팀 수(3~4개), 학생의 이름과 역할, 지원과 투표의 순서는 달라질 수 있음. 규칙은 이 공고에 적힌 그대로임

9. 모집 마감 때 어느 팀에도 들어가지 못해서 AI 배정을 받아야 하는 학생은 8명이 넘지 않도록 시나리오를 만들 예정임

6.3 감점

- 마감이 지나면 과제를 접수하지 않음
- 채점 항목에서 이미 점수를 잃는 일은 따로 감점하지 않음(같은 잘못을 두 번 깎지 않음). 감점 뒤 점수는 0점 아래로 내려가지 않음

구분	번호	사유	처리
제출	1	마감 뒤 저장소에 커밋함(Streamlit에 배포한 앱은 커밋하면 바로 바뀜)	-5
제출	2	배포 주소를 제출하지 않았거나 열리지 않는 주소를 제출함	시나리오 점수 0점
제출	3	11/3(화) 09:00까지 저장소를 공개로 바꾸지 않았고, 저장소 압축 파일도 제출하지 않음	시나리오 밖 점수 0점
제출	4	관리자 비밀번호를 제출하지 않았거나 틀림	시나리오 점수 0점
제출	5	배포한 앱이 제출한 저장소의 기본 브랜치(`master`나 `main`)가 아닌, 다른 저장소나 다른 브랜치의 코드로 실행됨	-3
제출	6	Streamlit Community Cloud가 아닌 곳에 배포함	-2
보안	7	키나 관리자 비밀번호를 코드에 적었거나 깃허브(커밋 기록 포함)에 올림	-3
보안	8	앱 화면에 오픈라우터 API 키, 또는 DB 비밀번호가 들어 있는 DB 주소(`DATABASE_URL`)를 출력하거나, 코드가 이 값들을 출력하게 되어 있음	-3
보안	9	명단 CSV, 코드, 문서, 화면 캡처에 가상 학생 대신 실제 사람(수강생 포함)의 이름이나 학번을 씀. `README.md`에 작성자로 자기 이름을 적는 것은 제외	-3
보안	10	비밀번호를 해시하지 않고 적힌 그대로 DB에 저장함	-2
보안	11	학생 계정으로 관리자 기능(명단 올리기, 마감, 배정)을 쓸 수 있음	-3
보안	12	비밀번호가 틀려도 로그인되거나, 학번만으로 남의 계정에 들어갈 수 있음	-3
DB와 AI	13	테이블 이름이 `tf_`로 시작하지 않음	-1
DB와 AI	14	무료 모델로 자동 연결하는 라우터(`openrouter/free`)를 씀	-2
채점 방해	15	2.7절 표에 적힌 화면 요소에 Streamlit `key`를 표와 다르게 지정했거나 지정하지 않아, 자동 채점 프로그램이 멈춤	-5
채점 방해	16	채점 기간에 DB(Supabase)가 일시정지되어 앱이 동작하지 않음. 앱이 잠든 것은 해당하지 않음(채점자가 깨움)	채점할 수 없으므로 시나리오 점수 0점
채점 방해	17	채점 기간에 관리자 비밀번호를 바꿈	시나리오 점수 0점
채점 방해	18	AI 배정이 10분이 넘도록 끝나지 않음(전체 대기 시간 제한이 없음)	채점자가 끊고, 그 시나리오의 배정 이후 채점 항목은 0점

구분	번호	사유	처리
정직성	19	연습용 시나리오의 이름이나 순서에 맞춰 결과를 미리 정해 둔 코드(예: 학생 이름을 보고 막음)	그 시나리오 점수 0점
정직성	20	`README.md`의 사례(7.1절 5~7항)의 화면이나 기록을 꾸며 냄	시나리오 밖 점수 0점
정직성	21	다른 수강생의 코드를 베낌	베낀 학생과 베끼게 한 학생 모두 과제 점수 0점

- 베낌 판정(21번): 같은 공고로 클로드가 만든 코드는 서로 비슷할 수 있으므로, 코드가 비슷하다는 것만으로는 베낌으로 보지 않음. 커밋 기록, `README.md`의 사례, 화면 문구 등이 겹쳐 베낀 것으로 의심되면 면담해 판정함
- 해시: 비밀번호를 원래대로 되돌릴 수 없는 다른 문자열로 바꿔 저장하는 것. 로그인할 때는 입력한 비밀번호를 같은 방법으로 바꿔 저장된 값과 비교함. DB가 새어 나가도 원래 비밀번호를 알 수 없음. 파이썬에서는 `bcrypt`로 할 수 있음

7. 제출물

- 제출 위치는 e-class의 이 과제 제출 칸이고, 기한은 11/2(월) 23:59임. 아래 제출물을 모두 냄

제출물	내용
깃허브 저장소	만드는 동안은 비공개 저장소로 둠. 마감 뒤 11/3(화) 09:00까지 공개로 바꿈(채점자가 코드와 `README.md`를 읽기 위함). 키, 관리자 비밀번호, 실제 개인정보가 없어야 함
배포 주소	Streamlit Community Cloud의 앱 주소
관리자 비밀번호	e-class 제출 칸에만 적음
보고서	저장소의 `README.md`. 7.1절의 양식
저장소 압축 파일	저장소를 압축한 zip 파일을 e-class에 올림(반드시 냄). 저장소를 읽을 수 없을 때를 위한 차선책임. 압축 파일에는 커밋 기록이 없으므로, 압축 파일로 채점하면 커밋을 근거로 든 사례(7.1절 6항)는 확인할 수 없어 점수를 받지 못할 수 있음. 클로드에게 다음과 같이 요청해 만듦: "이 저장소에서 커밋한 파일만 zip으로 압축해 줘. 파일 이름은 학번.zip으로 하고, 만든 뒤 zip 안에 .env가 없는지 확인해 줘." 폴더를 그대로 압축하면 숨은 파일인 `.env`가 들어갈 수 있으므로 그렇게 하지 않음

7.1 `README.md` 양식

1. 배포 주소. 예: `https://tf-abc123.streamlit.app`
2. PRD와 구현 계획의 위치. 예: `docs/PRD.md`, `docs/구현 계획.md`
3. 고른 해결 방법: 3장의 표에서 기능마다 쉬운 해결과 어려운 해결 가운데 무엇을 했는지. 예: "다른 사람의 변경 보기: 어려운 해결(`st.fragment`로 5초마다 갱신)"
4. 쓴 모델 3개의 이름. 예: "1차 `nvidia/nemotron-3-super-120b-a12b:free`, 2차 …, 3차 …"

5. 오픈라우터 AI 모델 호출 실패 사례 1건 이상과 그 화면. 예: "1차 모델이 120초 안에 응답하지 않아 2차 모델로 넘어감(화면 `img/ai-timeout.png`)", "1차 모델의 배정안에서 팀2에 웹 프로그래머가 없어 앱이 걸러 냄(화면 `img/ai-rejected.png`)"
6. 과제 해결 과정에서 클로드가 잘못된 추론이나 답변을 한 사례 1건 이상과, 그것을 알아챈 방법(화면이나 커밋을 근거로). 예: "클로드가 만든 투표 코드가 팀원 1명의 찬성만으로 승인했음. 중간 시나리오 6단계를 해 보다 알아챘고, 커밋 `a1b2c3d`에서 고침"

7. 본인이 제출한 앱에서 발견한 문제점이나 개선 필요 사항 1건 이상과, 그 문제를 발견한 연습용 시나리오의 단계와 화면. "문제 없음"은 인정하지 않음. 예: "부록 3(어려움)의 11단계에서 지원이 막히지 않음(필수 포함 역할을 맡을 학생의 자리를 남겨 두는 기능을 만들지 못함, 화면 `img/step11.png`)", "AI 배정 중에 화면이 2분 동안 멈춘 것처럼 보임"

8. 5~7항은 사례만 적으면 부분 점수를 받고, 근거(화면, 커밋, 시나리오 단계)까지 적으면 점수를 다 받음

8. 나눠 줄 자료

공고와 함께 e-class에 올림

자료	내용
`쉬움.csv`, `중간.csv`, `어려움.csv`	부록 연습용 시나리오의 명단. 단계와 기대 결과는 공고 부록에 있음
시연 동영상	https://youtu.be/ti-mDlKlfqg

부록. 연습용 시나리오 3개

- 이름의 약자: PL 기획자, WP 웹 프로그래머, FD 프론트엔드 디자이너, TT 테스터, RW 보고서 작성자, PT 발표자. 괄호 안 첫째가 주 역할이고 나머지가 보조 역할임. 예: 학생1(PL;RW)은 주 역할이 기획자, 보조 역할이 보고서 작성자임
- 명단 표의 보조 역할 "없음"은 CSV에서는 빈칸이고, 보조 역할 2개는 CSV에서 `;`로 구분함(2.6절)
- 할 일: 자기 앱에서 아래 단계를 위에서부터 직접 해 보고, 결과 안내의 첫 단어가 기대 결과와 같은지 확인함. 채점은 이와 난이도가 같은 다른 시나리오로 함
- 이름은 명단 CSV의 학생임. "막힘"은 앱이 지원을 받아들이지 않고 이유를 안내하는 것임
- "A가 팀1에 지원"은 A로 로그인해 팀1에 지원하는 것이고, "B가 A에 찬성"은 B로 로그인해 A의 지원에 찬성하는 것임
- 단계의 순서가 곧 시간의 순서임. 한 단계 안에 여러 일이 있으면 적힌 차례대로 함
- 지원이 막혔으면 그 지원에 대한 뒤의 투표는 건너뜀

부록 1. 쉬움: `쉬움.csv`, 팀 3개

명단

학번	이름	주 역할	보조 역할
2026101	학생1(PL;RW)	기획자	보고서 작성자
2026102	학생2(PL;PT)	기획자	발표자
2026103	학생3(PL;TT)	기획자	테스터
2026104	학생4(WP;FD)	웹 프로그래머	프론트엔드 디자이너
2026105	학생5(WP)	웹 프로그래머	없음

학번	이름	주 역할	보조 역할
2026106	학생6(WP;TT)	웹 프로그래머	테스터
2026107	학생7(FD;PT)	프론트엔드 디자이너	발표자
2026108	학생8(TT;RW;PT)	테스터	보고서 작성자, 발표자

단계

단계	하는 일	기대 결과
1	학생1(PL;RW)이 팀1에, 학생2(PL;PT)가 팀2에, 학생3(PL;TT)이 팀3에 지원	3명 모두 바로 승인
2	학생4(WP;FD)가 팀1에 지원, 학생1(PL;RW)이 학생4(WP;FD)에 찬성	승인
3	학생5(WP)가 팀2에 지원, 학생2(PL;PT)가 학생5(WP)에 찬성	승인
4	학생7(FD;PT)이 팀1에 지원, 학생1(PL;RW)이 학생7(FD;PT)에 찬성, 학생4(WP;FD)가 학생7(FD;PT)에 거부	거절
5	학생6(WP;TT)이 팀3에 지원, 학생3(PL;TT)이 학생6(WP;TT)에 찬성	승인
6	관리자가 마감하고 배정, 확정	남은 학생이 모두 배정되고, 모든 팀이 규칙을 지킴

부록 2. 중간: `중간.csv`, 팀 4개

명단

학번	이름	주 역할	보조 역할
2026201	학생1(PL;PT;TT)	기획자	발표자, 테스터
2026202	학생2(PL;TT)	기획자	테스터
2026203	학생3(PL;RW)	기획자	보고서 작성자
2026204	학생4(PL)	기획자	없음
2026205	학생5(WP;TT)	웹 프로그래머	테스터
2026206	학생6(WP;FD)	웹 프로그래머	프론트엔드 디자이너
2026207	학생7(WP)	웹 프로그래머	없음
2026208	학생8(WP;PT)	웹 프로그래머	발표자
2026209	학생9(FD;WP)	프론트엔드 디자이너	웹 프로그래머
2026210	학생10(TT;PL)	테스터	기획자
2026211	학생11(RW)	보고서 작성자	없음
2026212	학생12(PT;RW;FD)	발표자	보고서 작성자, 프론트엔드 디자이너

단계

단계	하는 일	기대 결과
1	학생1(PL;PT;TT)이 팀1에 지원	바로 승인
2	학생5(WP;TT)가 팀1에 지원, 학생1(PL;PT;TT)이 학생5(WP;TT)에 찬성	승인
3	학생2(PL;TT)가 팀1에 지원, 학생1(PL;PT;TT)과 학생5(WP;TT)가 학생2(PL;TT)에 찬성	막힘(주 역할 중복)
4	학생6(WP;FD)이 팀2에 지원	바로 승인
5	학생9(FD;WP)가 팀2에 지원, 학생6(WP;FD)이 학생9(FD;WP)에 찬성	승인
6	학생10(TT;PL)이 팀2에 지원, 학생6(WP;FD)이 찬성, 학생9(FD;WP)가 거부	거절
7	학생3(PL;RW)이 팀3에, 학생11(RW)이 팀4에 지원	2명 모두 바로 승인
8	학생12(PT;RW;FD)가 팀4에 지원(아무도 투표하지 않음). 이어서 팀3에도 지원	팀4 지원은 대기. 팀3 지원은 막힘
9	관리자가 마감	학생12(PT;RW;FD)의 지원은 거절. 학생3(PL;RW)과 학생11(RW)은 팀에서 빠짐
10	학생2(PL;TT)가 팀3에 지원	막힘(마감)
11	관리자가 배정, 확정	남은 학생이 모두 배정되고, 모든 팀이 규칙을 지킴

부록 3. 어려움: `어려움.csv`, 팀 4개

명단

학번	이름	주 역할	보조 역할
2026301	학생1(PL;RW)	기획자	보고서 작성자
2026302	학생2(PL;PT)	기획자	발표자
2026303	학생3(PL;TT)	기획자	테스터
2026304	학생4(WP;PL)	웹 프로그래머	기획자
2026305	학생5(WP;FD)	웹 프로그래머	프론트엔드 디자이너
2026306	학생6(WP;RW)	웹 프로그래머	보고서 작성자
2026307	학생7(FD;PT;TT)	프론트엔드 디자이너	발표자, 테스터
2026308	학생8(FD;TT)	프론트엔드 디자이너	테스터
2026309	학생9(FD;WP)	프론트엔드 디자이너	웹 프로그래머
2026310	학생10(TT;RW)	테스터	보고서 작성자
2026311	학생11(TT;PT)	테스터	발표자
2026312	학생12(TT;PL)	테스터	기획자
2026313	학생13(RW;PT;FD)	보고서 작성자	발표자, 프론트엔드 디자이너

학번	이름	주 역할	보조 역할
2026314	학생14(RW;TT)	보고서 작성자	테스터
2026315	학생15(PT;FD)	발표자	프론트엔드 디자이너
2026316	학생16(PT)	발표자	없음

단계

단계	하는 일	기대 결과
1	학생1(PL;RW)이 팀1에 지원	바로 승인
2	학생4(WP;PL)가 팀1에 지원, 학생1(PL;RW)이 학생4(WP;PL)에 찬성	승인
3	학생7(FD;PT;TT)이 팀1에 지원, 학생1(PL;RW)과 학생4(WP;PL)가 학생7(FD;PT;TT)에 찬성	승인
4	학생2(PL;PT)가 팀2에 지원	바로 승인
5	학생8(FD;TT)이 팀2에 지원, 학생2(PL;PT)가 학생8(FD;TT)에 찬성	승인
6	학생5(WP;FD)와 학생6(WP;RW)이 차례로 팀2에 지원	2명 모두 대기
7	학생2(PL;PT)와 학생8(FD;TT)이 학생5(WP;FD)에 찬성	승인
8	학생2(PL;PT), 학생8(FD;TT), 학생5(WP;FD)가 학생6(WP;RW)에 찬성	거절(승인 직전 검사)
9	학생11(TT;PT)이 팀3에 지원	바로 승인
10	학생15(PT;FD)가 팀3에 지원, 학생11(TT;PT)이 학생15(PT;FD)에 찬성	승인
11	학생14(RW;TT)가 팀3에 지원, 학생11(TT;PT)과 학생15(PT;FD)가 학생14(RW;TT)에 찬성	막힘(필수 포함 역할 자리)
12	학생10(TT;RW)이 팀4에 지원	바로 승인
13	학생13(RW;PT;FD)이 팀1에 지원, 학생1(PL;RW)만 학생13(RW;PT;FD)에 찬성	대기
14	관리자가 마감	학생13(RW;PT;FD)의 지원은 거절. 학생10(TT;RW)은 팀에서 빠짐
15	관리자가 배정, 확정	남은 학생이 모두 배정되고, 모든 팀이 규칙을 지킴